GEOMETRIC KNOWLEDGE SPACES, ATTENTION PAIRING AND ARCHITECTURES

SHANG XU

Abstract. In this paper, we propose a geometric approach to deriving attention mechanisms and exploring potential new transformer architectures. We begin by formalizing the notions of knowledge worlds and attention pairings, then demonstrate how this framework naturally leads to the architectures of standard attention, causal attention, linear attention, tensor attention, and recurrent neural networks. Finally, we introduce several new geometric spaces that may inspire novel neural network architectures.

Contents

1. Introduction 1
2. Formalization of Characters, Knowledge and Intelligence and Attention 2
2.1. An intuitive abstraction of the axioms 3
2.2. A categorical formalization 4
2.3. Examples 5
3. From the formalization to architectures 5
3.1. Bilinear pairing and attention 5
3.2. Matrix trace pairing and linear attention 7
3.3. Tensor-valued knowledge and natural attention pairings 8
3.4. Fixed-dimensional knowledge worlds and recurrent neural networks 9
4. Geometric spaces that lead to new designs 12
4.1. Further geometric knowledge worlds 12
References 14

1. Introduction

Neural network architectures embody choices about how information is represented, accumulated, and evaluated. Recurrent neural networks process sequential inputs by repeatedly updating a hidden state, allowing information from earlier inputs to influence later computations [SMH11]. Attention mechanisms instead provide a way to retrieve and combine information according to its relevance to a query [BCB15, VSP+17]. These mechanisms are usually presented through explicit computational formulas, involving matrix multiplication, nonlinear activation, normalization, and aggregation. Such formulas specify how a model operates, but they also invite a structural question: what mathematical objects and operations underlie these computations, and how might alternative choices lead to different architectures?

The use of geometry to understand and design neural networks has an established foundation. Geometric deep learning emphasizes how the structure and symmetries of a learning problem can guide the construction of architectures, bringing together methods for grids, groups, graphs, and manifolds within a common perspective [BBCV21]. The study of permutation-invariant learning provides a particularly concrete example: the requirement that a function be insensitive to input order leads to structured architectures for processing sets [ZKR+17]. These developments illustrate the value of treating mathematical structure as a source of architectural constraints and design principles. Complementing this perspective, we focus on the relationship between the space in which information is stored, the operations by which it is accumulated, and the pairings through which it is evaluated.

In this paper, we introduce a geometric framework organized around the notions of a *knowledge world* and an *intelligence*. A character space A represents elementary units of encoded information, while a sequence of spaces $(A_n)_{n\geq 0}$ describes the possible knowledge states after successive inputs. Update maps specify how a new character is incorporated into an existing state. An intelligence assigns to each state a function that evaluates characters in a stage-dependent evaluation space V_n. In a concrete setting, these

structures are expressed by maps

$$\mu_n : A \times A_n \to A_{n+1}, \qquad B_n : A_n \times A \to V_n,$$

The spaces V_n may have several coordinates, corresponding to different evaluation criteria, and their dimensions may vary with the stage. We call B_n an *attention pairing*, using the term "pairing" for a structured two-input evaluation map without requiring it to be bilinear.

An important feature of the framework is the distinction between evaluating information and using that evaluation in a computation. A pairing may produce compatibility scores, a proposed memory increment, or a candidate hidden representation. Additional operations determine how these evaluations affect the model: they may be normalized, used to rescale and aggregate stored information, or incorporated into a subsequent knowledge state. For example, scaled dot-product attention combines a query–key pairing with softmax normalization and weighted aggregation of values [VSP+17]. In our formulation, these are separate choices rather than inseparable parts of the definition of intelligence. Likewise, learned encoders may determine how input data enter the character space, while the pairing on that space can be fixed by its geometric structure. This separation makes it possible to vary the representation, evaluation, and update mechanisms independently, subject to their compatibility.

We illustrate the framework by realizing the central computations of several established architectures. Sequence-valued knowledge states equipped with inner-product pairings recover dot-product attention. Restricting evaluation and normalization to prefix states gives causal attention without changing the underlying pairing. Additive accumulation of rank-one contributions yields a knowledge world whose normalized readout recovers positive-feature linear attention, consistent with its recurrent formulation [KVPF20]. Low-rank tensor-valued characters, together with contractions that retain the separate attention heads, provide a geometric realization of Tensor Product Attention [ZLY+25]. Finally, state spaces that stabilize in dimension support recurrent architectures: bilinear evaluations can generate accumulated interaction memories, while nonlinear evaluations can recover the affine-plus-activation recurrence of a vanilla RNN [SMH11]. These examples concern the core computational mechanisms; their incorporation into complete models requires further choices of parametrization, layer composition, and output maps.

The framework also makes several structural distinctions explicit. We call a knowledge world *abelian* when incorporating two characters in either order gives the same state. Starting from a fixed initial state, this condition makes the final accumulated knowledge insensitive to permutations of the input characters. We separately consider a *placeholder-free* condition, which requires insertion of a distinguished null character to commute with other updates. This condition does not, by itself, require null insertion to act as the identity. Causality is different from both properties: it concerns which information is available when an output is computed. An abelian accumulation rule can therefore support causal computations when each output is evaluated on the corresponding prefix. Distinguishing properties of storage, evaluation, and information availability helps clarify which architectural choices are responsible for particular computational behaviors. More generally, topology, differential geometry, and algebraic geometry offer spaces and operations that can be investigated within this framework. The aim is not merely to translate existing architectures into geometric terminology, but to make geometric constructions available as inputs to the design process.

This perspective separates mathematical specification from computational and empirical validation. A geometric system satisfying the proposed axioms does not automatically determine a unique architecture, nor does it guarantee efficient implementation or effective learning. Parametrization, numerical stability, computational cost, and the interaction between layers remain essential design questions. Nevertheless, making the underlying structures explicit provides a way to formulate these questions and to compare candidate designs. We begin with an accessible description of the framework and its categorical formalization, then examine attention and recurrent architectures as examples, and finally explore geometric constructions that may lead to new computational attention layers.

Acknowledgments. Thanks Ziang Li and Pengcheng Wang for very useful discussions.

AI disclosures. In this first version of paper, ChatGPT is used for literature searches, computation check and language polishing. The ideas and mathematical interference are original and the author takes full responsibility for its correctness.

2. Formalization of Characters, Knowledge and Intelligence and Attention

In this section we explain our abstraction of learning theory to both computer scientists and mathematicians. Subsection 2.1 is for computer scientists to quickly understand our setting, and 2.2 is a categorical formalism for mathematicians.

2.1. **An intuitive abstraction of the axioms.** We begin by introducing a mathematical framework for describing how information is accumulated, processed, and evaluated. The basic objects are *characters*, *knowledge states*, *processes*, and *intelligence*. We first explain these notions in the language of Euclidean spaces, so that the framework can be understood without any background in abstract mathematics.

Let $A \subseteq \mathbb{R}^d$ be a space of *characters*, together with a distinguished element $0 \in A$, called the *null character*. A character represents an elementary piece of information. For example, in a language model, a character may represent a token embedding; in computer vision, it may represent a feature vector extracted from an image patch. Thus, the word "character" should be understood as a general unit of information, rather than necessarily a letter or symbol.

For each nonnegative integer n, let $A_n \subseteq \mathbb{R}^{d_n}$ be a space of *knowledge states*. Intuitively, A_n represents all possible internal states of a system after receiving n characters. These spaces need not be Euclidean spaces themselves: they may be submanifolds or other geometric subsets of $\mathbb{R}^{d_n}$. Moreover, the dimensions d_n are allowed to vary with n. Of course, we would like to let $A_1 = A$ so that the first stage of knowledge is just our character space. Now, to connect those stages, there should be a family of maps

$$\mu_n : A \times A_n \to A_{n+1},$$

which describe how knowledge is updated upon receiving a new character. More precisely, given a current knowledge state $x \in A_n$ and a character $t \in A$, the new state is

$$x' = \mu_n(t, x) \in A_{n+1}.$$

Starting from an initial state $x_0 \in A_0$, a sequence of characters $t_1, \ldots, t_N$ therefore generates a sequence of knowledge states recursively:

$$x_{n+1} = \mu_n(t_{n+1}, x_n).$$

This is a formalization of how information unit establish the knowledge.

We are particularly interested in two properties of these updates. First, we call the system *placeholder-free* if

$$\mu_{n+1}(0, \mu_n(t, x)) = \mu_{n+1}(t, \mu_n(0, x))$$

for every $x \in A_n$ and $t \in A$. In other words, inserting a null character before or after another character leads to the same knowledge state. Notice that this condition does not require the null character to have no effect; it only requires its update to commute with other character updates.

Second, we call the system *abelian* if

$$\mu_{n+1}(s, \mu_n(t, x)) = \mu_{n+1}(t, \mu_n(s, x))$$

for every $x \in A_n$ and $s, t \in A$. This means that the order in which two characters are incorporated does not affect the resulting knowledge state. Consequently, the final state obtained from a fixed initial state depends only on the multiset of input characters, rather than their order. A simple example is obtained by taking $A = A_n = \mathbb{R}^{d \times d}$ for all n, with

$$\mu_n(t, x) = x + t.$$

Since matrix addition is commutative, this system is abelian. More generally, the maps μ_n may involve nonlinear transformations or geometric constraints, and need not satisfy the abelian property. Abelian systems are natural candidates for tasks involving unordered information, such as learning from sets of feature vectors.

Having established the knowledge space, we now introduce two additional structures.

(1) A *process* is a sequence of maps

$$p_n : A_n \to A.$$

This can be understood as given a sentence with n characters, we can determine the next in a machinery manner. Composing with the updating map, we get the generating map P_n:

$$P_n : A_n \xrightarrow{p_n \times id} A \times A_n \xrightarrow{\mu_n} A_{n+1}$$

One may think of this as intelligence because it automatically feedback once you have some input. However, an intelligence should be able to 'see' all possible results and 'determine' which to choose. This is formalized as following.

(2) For each n, we obtain an evaluation space V_n, which one can consider as a multi-dimensional evaluation criteria at the stage n, and we may initially take to be $\mathbb{R}^{k_n}$, where k_n is the number of factors we want to consider. Let $\mathrm{Fun}_n(A)$ denote the space of functions from A to V. An *intelligence* is a sequence of maps

$$w_n : A_n \to \mathrm{Fun}_n(A).$$

Thus, each knowledge state $x \in A_n$ determines an evaluation function

$$w_n(x) : A \to V_n.$$

Mathematically, one usually has an adjunction to the functor $[A, -]$, namely $A \otimes -$, where $\otimes$ could be another operation between spaces that behaves like the tensor product of vector spaces. The key is we want

$$\mathrm{Map}(A_n, \mathrm{Fun}_n(A)) = \mathrm{Map}(A_n, \mathrm{Map}(A, V_n)) = \mathrm{Map}(A_n \otimes A, V_n)$$

To understand this intuitively, if we denote $B_n(x,t) = w_n(x)(t)$ for $x \in A_n$ and $t \in A$, we see that a map from A_n to $\mathrm{Fun}_n(A)$ gives a function (a map to V_n) on $A_n \times A$. If spaces and maps are linear, this reduces to a map from $A_n \otimes A$ to V_n. Different choices of structured maps (such as continuous, differentiable, or polynomial) could lead to different constructions of the Map space/tensor constructions, so we need to formalize this here. mathematically, we call a function on $A_n \times A$ a *pairing* on A_n and A. For this specific structure, we call w_n the *attention pairing* at the stage n, due to its close relationship with the attention mechanism which we will explain in the following sections.

2.2. **A categorical formalization.** This subsection is for mathematics For mathematicians, the following categorical formulation may be preferable, as it allows the spaces above to be replaced by more general geometric or algebraic objects.

Fix a locally small category of spaces $\mathcal{C}$ with a terminal object T, and suppose that binary products exist in $\mathcal{C}$. A *space of characters* is a pointed object $(A, 0)$, where $A \in \mathcal{C}$ and $0 : T \to A$ is a distinguished morphism called the *null character*. A *knowledge world* with respect to A is an $\mathbb{N}$-indexed sequence of objects $(A_n)_{n\in\mathbb{N}}$ with $A_1 = A$ equipped with morphisms

$$\mu_n : A \times A_n \to A_{n+1}.$$

For any character $t : T \to A$, we denote by $\mu_n^t : A_n \to A_{n+1}$ the morphism induced by t, defined as the composition

$$\mu_n^t : A_n \cong T \times A_n \xrightarrow{t\times \mathrm{id}_{A_n}} A \times A_n \xrightarrow{\mu_n} A_{n+1}.$$

A knowledge world is said to be *placeholder-free* if

$$\mu_{n+1}^0 \circ \mu_n^t = \mu_{n+1}^t \circ \mu_n^0$$

for every character $t : T \to A$ and every $n \in \mathbb{N}$. Similarly, it is called *abelian* if

$$\mu_{n+1}^s \circ \mu_n^t = \mu_{n+1}^t \circ \mu_n^s$$

for all characters $s, t : T \to A$ and every $n \in \mathbb{N}$. In particular, every abelian knowledge world is placeholder-free.

Having established the notion of a knowledge world, we now define processes and intelligence on this structure. A *process* is an $\mathbb{N}$-indexed sequence of morphisms $(p_n)_{n\in\mathbb{N}}$, where

$$p_n : A_n \to A_{n+1}.$$

Next, fix an evaluation object $V \in \mathcal{C}$. We wish to define the object of *functions* on A, denoted by $\mathrm{Fun}_n(A)$, such that the the morphism from objects in $\mathcal{C}$ to $\mathrm{Fun}_n(A)$ makes sense. If objects in $\mathcal{C}$ are sets, we may simply take $\mathrm{Fun}_n(A) = \mathrm{Hom}_{\mathcal{C}}(A, V)$. More generally, we suppose that $\mathcal{C}$ admits an internal mapping object $[A, V]$, characterized by natural bijections

$$\mathrm{Hom}_{\mathcal{C}}(X, [A, V]) \cong \mathrm{Hom}_{\mathcal{C}}(X \times A, V)$$

for every $X \in \mathcal{C}$. In this case, we define

$$\mathrm{Fun}_n(A) := [A, V].$$

Under either assumption, an *intelligence* on the knowledge world is an $\mathbb{N}$-indexed sequence of morphisms

$$w_n : A_n \to \mathrm{Fun}_n(A).$$

Equivalently, by the defining property of the internal mapping object, each w_n corresponds to an evaluation morphism

$$\widetilde{w}_n : A_n \times A \to V.$$

This latter formulation remains meaningful even when the internal mapping object does not exist, and may therefore be used as a more general definition of intelligence.

Thus, a knowledge world, together with a process and an intelligence, provides a categorical description of information accumulation, state evolution, and evaluation, without imposing any particular geometric structure on the underlying objects.

2.3. **Examples.** In this section, we provide some examples of filtered geometric spaces satisfying our axioms, and we will talk about the concrete industry design derived by these structures in the next section.

Example 2.1. vector space and bilinear form Let $A = \mathbb{R}^d$ be the character space, and take

$$A_n = A^{\oplus n} = \mathbb{R}^{d\times n}.$$

A knowledge state $X = (v_1, \dots, v_n) \in A_n$ records a sequence of n characters, each already encoded as a vector in $\mathbb{R}^d$. We regard these vectors as the columns of X. The knowledge-update map appends a new character:

$$\mu_n : A \times A_n \to A_{n+1}, \qquad \mu_n(v, (v_1, \dots, v_n)) = (v_1, \dots, v_n, v).$$

Take $V_n = \mathbb{R}^n$. The inner product defines a linear intelligence

$$w_n : A_n \to \mathrm{Hom}_{\mathbb{R}}(A, V_n), \qquad w_n(v_1, \dots, v_n) = \big(\langle v_1, -\rangle, \dots, \langle v_n, -\rangle\big).$$

This leads to the usual attention and casual attention, we will explain in 3.1 and 3.1.1.

Example 2.2. Let $A = A_1 = V$ and $A_n = V \oplus U$ for some real vector space V, U. Choose an initial state h_0, the update is

$$\mu_0(a, *) = a, \quad \mu_1(a, s) = \big(a, F_\theta(F_\theta(h_0, s), a)\big), \quad \mu_n(a, (s, h)) = \big(a, F_\theta(h, a)\big),\ n \geq 2$$

Take $V_n = U$, an intelligence is defined by

$$w_0(*)(a) = F_\theta(h_0, a), \quad w_1(s)(a) = F_\theta(F_\theta(h_0, s), a), \quad w_n(s, h)(a) = F_\theta(h, a),\ n \geq 2$$

This would lead to RNN as we will explain in 3.4.

Example 2.3. Fix an ambient matrix space $\mathfrak{gl}(N, \mathbb{R}) = \mathbb{R}^{N\times N}$, let

$$A_n = \{M \in \mathfrak{gl}(N, \mathbb{R}) | \operatorname{rank} M \leq n\}$$

We have an update

$$\mu : A_n \times A_1 \to A_{n+1}, \quad \mu(M, N) = M + N$$

An intelligence is induced by the attention pairing

$$\langle M, N\rangle = \mathrm{Tr}(MN^T)$$

This leads to *linear attention* as explained in 3.2

There are also other interesting examples, for example, the tensor space in 3.3 that leads to *tensor attention*, the projective spaces, Grassmannians, Lie algebra orbits.

3. From the formalization to architectures

In this section, we specify how the geometric constructions of the knowledge world and intelligence in last section's examples lead to the actual architectures in industry. Our examples cover attention, casual attention, linear attention, tensor attention and vanilla/multiplicative RNN, showing the strong potential ability of our formalization in architectural designs.

3.1. **Bilinear pairing and attention.** Let $A = \mathbb{R}^d$ be the character space, and take

$$A_n = A^{\oplus n} = \mathbb{R}^{d\times n}.$$

A knowledge state $X = (v_1, \dots, v_n) \in A_n$ records a sequence of n characters, each already encoded as a vector in $\mathbb{R}^d$. We regard these vectors as the columns of X. The knowledge-update map appends a new character:

$$\mu_n : A \times A_n \to A_{n+1}, \qquad \mu_n(v, (v_1, \dots, v_n)) = (v_1, \dots, v_n, v).$$

3.1.1. *Bilinear intelligence and attention.* Take the evaluation space to be $V_n = \mathbb{R}^n$. The Euclidean inner product defines a linear intelligence

$$w_n : A_n \to \mathrm{Hom}_{\mathbb{R}}(A, V_n), \qquad w_n(v_1, \dots, v_n) = \big(\langle v_1, -\rangle, \dots, \langle v_n, -\rangle\big).$$

Each coordinate evaluates the compatibility of a character with one of the stored tokens. Thus, the number of evaluation criteria grows with the amount of stored information.

The corresponding attention pairing is

$$B_n(X, v) = w_n(X)(v) = \big(\langle v_1, v\rangle, \dots, \langle v_n, v\rangle\big) = X^{\mathsf{T}} v.$$

This map is linear in X and linear in v, and therefore corresponds to a linear map $A_n \otimes_{\mathbb{R}} A \to V_n$. The central evaluation structure is precisely this bilinear pairing: a knowledge state evaluates a character by comparing it with each character already stored.

We may now introduce a normalization on the evaluation space. For $n \geq 1$, define

$$\mathrm{SoftMax}_n : \mathbb{R}^n \to \mathbb{R}^n, \qquad (z_1, \dots, z_n) \longmapsto \left(\frac{e^{z_1}}{\sum_{i=1}^n e^{z_i}}, \dots, \frac{e^{z_n}}{\sum_{i=1}^n e^{z_i}}\right).$$

Its output has positive coordinates summing to one, so the normalized evaluations can be interpreted as weights assigned to the stored tokens.

Because softmax is nonlinear, postcomposition with it does not preserve the space of linear maps. We therefore regard $\mathrm{Hom}_{\mathbb{R}}(A, V_n)$ as a subspace of the larger function space $\mathrm{Fun}_n(A) := C(A, V_n)$. Softmax then induces a map

$$S_n : \mathrm{Fun}_n(A) \to \mathrm{Fun}_n(A), \qquad \phi \longmapsto \mathrm{SoftMax}_n \circ \phi.$$

Applying this operation to the intelligence, together with a scaling factor $1/\sqrt{d}$, gives

$$\widehat{w}_n(X)(v) = \mathrm{SoftMax}_n\left(\frac{w_n(X)(v)}{\sqrt{d}}\right) = \mathrm{SoftMax}_n\left(\frac{X^{\mathsf{T}} v}{\sqrt{d}}\right).$$

This normalization is an additional architectural choice, rather than a requirement of our framework. Exponentiation makes the weights positive, while division by their sum normalizes them. These operations introduce a nonlinear transformation of the evaluation scores. The underlying intelligence w_n is linear and its pairing B_n is bilinear, whereas the normalized evaluation $\widehat{w}_n$ is no longer bilinear.

The coordinates of $\widehat{w}_n(X)(v)$ are exactly the attention weights in the *scaled dot-product attention* mechanism described in [VSP+17, Section 3.2.1]: the character v is the *query*, and the stored vectors $v_1, \dots, v_n$ are the *keys*. To obtain the full attention output, associate a *value* vector $u_j \in \mathbb{R}^r$ with each key and form

$$\mathrm{Attn}(v; X, U) = \sum_{j=1}^n \big[\widehat{w}_n(X)(v)\big]_j u_j = U\,\mathrm{SoftMax}_n\left(\frac{X^{\mathsf{T}} v}{\sqrt{d}}\right),$$

where $U = (u_1, \dots, u_n) \in \mathbb{R}^{r \times n}$. In the simplest case, the stored vectors also serve as values, so $U = X$.

3.1.2. *Alternative choice: Causal Attention.* Let's continue working on the knowledge world above and introduce *causality*, we require the output at position i to use only the prefix state $X_i = (v_1, \dots, v_i)$, rather than the entire sequence X_N. The input vectors themselves are assumed not to contain information from future tokens. Evaluating the current character v_i and normalizing on V_i gives

$$\widehat{w}_i(X_i)(v_i) = \mathrm{SoftMax}_i\left(\frac{B_i(X_i, v_i)}{\sqrt{d}}\right).$$

Explicitly, the weight assigned to position $j \leq i$ is

$$\alpha_{ij} = \frac{\exp(\langle v_j, v_i\rangle/\sqrt{d})}{\sum_{\ell=1}^i \exp(\langle v_\ell, v_i\rangle/\sqrt{d})}.$$

The crucial point is that both the available scores and the normalizing denominator involve only the prefix. Softmax remains an additional nonlinear normalization of the bilinear evaluations; causality determines which evaluations may participate.

Associate a value vector $u_j = W_V v_j \in \mathbb{R}^m$ with each stored token. The retrieved output is

$$y_i = \sum_{j=1}^i \alpha_{ij} u_j.$$

This is *causal scaled dot-product self-attention*, as used in the Transformer decoder of [VSP+17, Sections 3.2.1 and 3.2.3], in the special case where the token vectors themselves serve as queries and keys.

The usual trainable version replaces $\langle v_j, v_i\rangle$ by $\langle W_K v_j, W_Q v_i\rangle$ and scales by the square root of the query–key dimension.

Causality is distinct from our commutativity axioms. The append-based world is neither abelian nor placeholder-free, since changing the order of appended characters changes the stored sequence. Nevertheless, its attention outputs are causal: appending future characters does not change the outputs already determined by a prefix.

Thus, causal attention retains the same bilinear intelligence as ordinary attention, but restricts each evaluation and its normalization to the knowledge available at that position.

Remark 3.1 (A query–key–value knowledge world). A more explicit formulation takes

$$A = \mathbb{R}^q \oplus \mathbb{R}^k \oplus \mathbb{R}^v \cong \mathbb{R}^{q+k+v}, \qquad A_n = A^{\oplus n}.$$

Each character $a = (Q, K, U)$ already consists of its query, key, and value components; these are coordinates of the character itself, not additional representations introduced by the intelligence. The knowledge-update maps remain concatenation.

For the Euclidean pairing, assume $q = k$. Given a state $X = (a_1, \dots, a_n)$, where $a_j = (Q_j, K_j, U_j)$, define

$$B_n(X, a) = \big(\langle K_1, Q\rangle, \dots, \langle K_n, Q\rangle\big).$$

Applying softmax after scaling by $1/\sqrt{k}$ gives weights for aggregating the stored value components U_j.

The single-vector presentation above is an intuitive specialization in which the query and key components coincide; the value component may also be identified with them. Keeping these components separate gives the query–key–value formulation used in the tensor-valued construction. Causal attention uses the same pairing, restricted to prefix states.

3.2. **Matrix trace pairing and linear attention.** We now consider a knowledge world in which information is accumulated by adding rank-one matrices, rather than by storing an entire sequence.

Fix positive integers r, m, and define the character space

$$A = \{0\} \cup \big\{a(u, v) := [u \mid uv^{\mathsf{T}}] : u \in \mathbb{R}^r_{>0},\ v \in \mathbb{R}^m\big\} \subseteq \mathbb{R}^{r\times(m+1)}.$$

Here $\mathbb{R}^r_{>0}$ denotes the vectors with strictly positive coordinates. A nonzero character encodes a positive feature vector u together with an information vector v, while the zero matrix is the null character. Each nonzero character has rank one.

For each $n \geq 0$, define

$$A_n = \{0\} \cup \Big\{H = [z \mid M] \in \mathbb{R}^{r\times(m+1)} : z \in \mathbb{R}^r_{>0},\ \operatorname{rank} H \leq n\Big\}.$$

Then

$$A_0 = \{0\}, \qquad A_1 = A, \qquad A_n \subseteq A_{n+1}.$$

The rank bound expresses the fact that a state obtained from n characters is a sum of at most n rank-one contributions; its rank need not increase at every update.

Define the knowledge-update maps by addition:

$$\mu_n : A \times A_n \to A_{n+1}, \qquad \mu_n(a, H) = H + a.$$

These maps are well-defined because

$$\operatorname{rank}(H + a) \leq \operatorname{rank} H + \operatorname{rank} a \leq n + 1.$$

More generally, $A_i + A_j \subseteq A_{i+j}$. Writing $H_n = [z_n \mid M_n]$, the update associated with $a(u, v)$ is explicitly

$$z_{n+1} = z_n + u, \qquad M_{n+1} = M_n + uv^{\mathsf{T}}.$$

Starting from $H_0 = 0$, the characters $a(u_1, v_1), \dots, a(u_n, v_n)$ therefore produce

$$H_n = [z_n \mid M_n], \qquad z_n = \sum_{j=1}^{n} u_j, \qquad M_n = \sum_{j=1}^{n} u_j v_j^{\mathsf{T}}.$$

Thus, the history is compressed into a matrix of fixed size.

This knowledge world is abelian, since

$$\mu^b_{n+1}\big(\mu^a_n(H)\big) = H + a + b = H + b + a = \mu^a_{n+1}\big(\mu^b_n(H)\big).$$

It is also placeholder-free. Indeed, μ^0_n is precisely the inclusion $A_n \hookrightarrow A_{n+1}$: a null character leaves the stored matrix unchanged. Consequently, the final memory depends only on the collection of encoded characters, not on their order. This does not imply that the intermediate prefix states are independent of order.

To define intelligence, take $V_n = \mathbb{R}^2$. For a state $H = [z \mid M] \in A_n$ and a character $b = [q \mid N] \in A$, we set the following pairing

$$w_n(H)(b) = B_n(H, b) = \big(\langle M, N\rangle_{\mathrm{F}}, z^{\mathsf{T}} q\big),$$

where

$$\langle M, N\rangle_{\mathrm{F}} = \mathrm{tr}(M^{\mathsf{T}} N)$$

is the Frobenius inner product. This defines an intelligence

$$w_n : A_n \to \mathrm{Fun}_n(A), \qquad \mathrm{Fun}_n(A) = C(A, \mathbb{R}^2).$$

The attention pairing B_n is the restriction of a bilinear map on the ambient matrix spaces. Its first coordinate evaluates compatibility, while its second records the quantity used for normalization.

As in the previous example, normalization is an additional operation on the evaluation space. Since $z^{\mathsf{T}} q \geq 0$, we may fix $\varepsilon > 0$ and apply

$$\nu_\varepsilon : \mathbb{R} \times \mathbb{R}_{\geq 0} \to \mathbb{R}, \qquad (s, c) \longmapsto \frac{s}{\varepsilon + c}.$$

This gives a scalar-valued normalized intelligence

$$\widehat{w}_n(H)(b) = \frac{\langle M, N\rangle_{\mathrm{F}}}{\varepsilon + z^{\mathsf{T}} q}.$$

The positive constant ε keeps this expression defined at the null state and null character. Since the formula is independent of the stage, inserting a null character also leaves all evaluations unchanged.

To interpret this intelligence, write a nonzero character as

$$b = a(q, v) = [q \mid q v^{\mathsf{T}}].$$

Then

$$\widehat{w}_n(H)(b) = \frac{v^{\mathsf{T}} M^{\mathsf{T}} q}{\varepsilon + z^{\mathsf{T}} q} = \left\langle \frac{M^{\mathsf{T}} q}{\varepsilon + z^{\mathsf{T}} q}, v \right\rangle.$$

Thus, for each positive query vector q, the normalized intelligence is represented by the retrieved vector

$$y_n(q) = \frac{M_n^{\mathsf{T}} q}{\varepsilon + z_n^{\mathsf{T}} q} = \frac{\sum_{j=1}^n (q^{\mathsf{T}} u_j) v_j}{\varepsilon + \sum_{j=1}^n q^{\mathsf{T}} u_j}.$$

Its evaluation on $a(q, v)$ is simply $\langle y_n(q), v\rangle$.

This is the positive-feature *linear attention* construction of [KVPF20, Sections 3.2–3.4], with an added regularization constant ε. Here q and u_j are the query and key features, and v_j are the values. For a nonzero memory and a positive query, setting $\varepsilon = 0$ in the readout gives positive attention weights summing to one. Unlike softmax attention, the similarities are $q^{\mathsf{T}} u_j$, rather than exponentials of dot products; the two mechanisms are not generally identical.

The computational advantage is that the sums z_n and M_n can be maintained incrementally and reused for every query. Each update and readout requires $O(rm)$ operations, so N updates and N readouts require $O(Nrm)$ operations, with $O(rm)$ storage for the running state. Thus, "linear" refers to the dependence on sequence length, not to linearity of the normalized output.

In our framework, this construction combines a rank-filtered knowledge world, abelian and placeholder-free updates, an ambient bilinear attention pairing, and a nonlinear normalization of its evaluations.

3.3. **Tensor-valued knowledge and natural attention pairings.** Fix a number of heads h and a feature dimension r, and consider the tensor space

$$E = \mathbb{R}^h \otimes \mathbb{R}^r \cong \mathbb{R}^{h \times r}.$$

For a positive integer R, let

$$\mathcal{D}_R = \{Z \in E : \mathrm{rank}\, Z \leq R\}.$$

Thus, $\mathcal{D}_R$ consists of tensors expressible as sums of at most R rank-one tensors.

Choose rank bounds R_Q, R_K, R_U, and define the character space by

$$A = \mathcal{D}_{R_Q} \times \mathcal{D}_{R_K} \times \mathcal{D}_{R_U}.$$

A character $a = (Q, K, U)$ already contains its query, key, and value tensors. These tensors are part of the encoded character itself, rather than representations constructed by the intelligence. The null character is $(0, 0, 0)$.

Take $A_n = A^n$, with A_0 consisting of the empty sequence, and define the knowledge-update maps by appending:

$$\mu_n(a, (a_1, \ldots, a_n)) = (a_1, \ldots, a_n, a).$$

Since the rank-constrained character space need not be a vector space, A^n here denotes a Cartesian product.

The tensor space carries a natural headwise pairing by

$$\kappa(\alpha \otimes \beta, \gamma \otimes \delta) = (\alpha \odot \gamma)\langle \beta, \delta \rangle,$$

where $\odot$ denotes coordinatewise multiplication. Bilinearity then determines the pairing on arbitrary tensors. In coordinates, this map is:

$$\kappa : E \times E \to \mathbb{R}^h, \qquad [\kappa(Q,K)]_\ell = \sum_{b=1}^{r} Q_{\ell b} K_{\ell b}.$$

Now, take the evaluation space to be

$$V_n = (\mathbb{R}^h)^n \cong \mathbb{R}^{h \times n}.$$

For a knowledge state $X = (a_1, \ldots, a_n)$, with $a_j = (Q_j, K_j)$, and a character $a = (Q, K, U)$, define

$$w_n(X)(a) = B_n(X, a) = \big(\kappa(Q, K_1), \ldots, \kappa(Q, K_n)\big).$$

This gives an intelligence

$$w_n : A_n \to \mathrm{Fun}_n(A), \qquad \mathrm{Fun}_n(A) = C(A, V_n).$$

Each evaluation coordinate records the compatibility between the query of a and the key of a stored character, within a particular head.

The pairing is the restriction of a bilinear map on the ambient tensor spaces. No additional trainable feature maps are required to define it: the tensor structure is already present in the character space.

As before, introduce a nonlinear normalization on the evaluation space:

$$\widehat{w}_n(X)(a) = \mathrm{SoftMax}_{\mathrm{row}}\left(\frac{B_n(X,a)}{\sqrt{r}}\right).$$

Softmax acts separately on the n evaluation coordinates of each head. Writing $P = \widehat{w}_n(X)(a)$, the retrieved tensor $Y(X,a) \in E$ is defined by

$$Y(X,a)_{\ell b} = \sum_{j=1}^{n} P_{\ell j} (U_j)_{\ell b}.$$

Thus, each head combines the corresponding rows of the stored value tensors according to its normalized evaluations.

This realizes the attention computation of *Tensor Product Attention* [ZLY+25, Section 3.1] directly on tensor-valued characters, before the final output projection. The low-rank constraints describe the geometry of the character space, while κ provides the evaluation pairing.

In the usual parametrized implementation, learned factor maps construct the tensors from input hidden vectors. In our formulation, these maps instead specify an encoder

$$e_\theta : \mathbb{R}^d \to A.$$

The resulting scores are obtained by evaluating

$$B_n\big((e_\theta(x_1), \ldots, e_\theta(x_n)), e_\theta(x)\big).$$

Choosing the encoder to be the usual tensor-factor construction recovers the corresponding scores and head outputs. The factors may also be retained as a compact representation for computing the contraction, without materializing the full tensors.

Consequently, the learned encoding and the intelligence are conceptually separate: learning determines how information is represented in the tensor-valued character space, while intelligence evaluates those representations through a fixed pairing. Normalization and value aggregation then turn these evaluations into an attention readout.

3.4. **Fixed-dimensional knowledge worlds and recurrent neural networks.** Let $V = \mathbb{R}^d$ and $U = \mathbb{R}^m$. Take the character space to be $A = V$, with null character 0, and consider the state spaces

$$A_0 = \{*\}, \qquad A_1 = V, \qquad A_n = V \oplus U \quad (n \geq 2).$$

Unlike the sequence spaces $V^{\oplus n}$, these spaces have a fixed dimension after the second stage. New information must therefore be incorporated into an existing representation, rather than stored by introducing new coordinates.

The state spaces alone do not determine a recurrent architecture. Different choices of intelligence and knowledge-update maps lead to different ways of processing information. We illustrate this with two constructions: a bilinear accumulation model and a vanilla recurrent neural network.

In both constructions, take the evaluation spaces to be $V_n = U$ and write

$$w_n : A_n \to \mathrm{Fun}_n(A), \qquad \mathrm{Fun}_n(A) = C(A, U).$$

Thus, an intelligence evaluates a candidate character by producing a vector in the memory space U. The knowledge updates below are explicitly constructed using these evaluations. This is a choice of architecture, rather than an additional requirement in the definition of intelligence.

3.4.1. *A bilinear recurrent architecture.* Choose a trainable bilinear map

$$\Gamma_\theta : V \times V \to U.$$

For $n \geq 2$, interpret a state $(s, c) \in V \oplus U$ as consisting of accumulated character information s and an interaction memory c. Define the intelligence by

$$\begin{aligned} w_0(*)(a) &= 0, \\ w_1(s)(a) &= \Gamma_\theta(s, a), \\ w_n(s, c)(a) &= \Gamma_\theta(s, a), \qquad n \geq 2. \end{aligned}$$

The corresponding attention pairing, for $n \geq 2$, is

$$B_n((s, c), a) = \Gamma_\theta(s, a).$$

It is bilinear in the knowledge state and the character. Its value represents the interaction between the accumulated character information and the new input, rather than the complete next memory state.

Define the knowledge-update maps by

$$\mu_0(a, *) = a, \qquad \mu_1(a, s) = \big(s + a, \Gamma_\theta(s, a)\big),$$

and, for $n \geq 2$, by

$$\mu_n(a, (s, c)) = \big(s + a, c + \Gamma_\theta(s, a)\big).$$

Equivalently, the intelligence determines the increment to the interaction memory:

$$\mu_n(a, (s, c)) = \big(s + a, c + w_n(s, c)(a)\big).$$

After identifying A_1 with $V \oplus \{0\} \subseteq H$, the evolution is described by the recurrence

$$s_{n+1} = s_n + a_{n+1}, \qquad c_{n+1} = c_n + \Gamma_\theta(s_n, a_{n+1}),$$

with $s_0 = 0$ and $c_0 = 0$. Bilinearity gives the explicit formulas

$$s_n = \sum_{j=1}^{n} a_j, \qquad c_n = \sum_{1 \leq j < k \leq n} \Gamma_\theta(a_j, a_k).$$

Thus, the first component stores a first-order summary of the inputs, while the second stores a summary of ordered pairwise interactions. The earlier character appears in the first argument of Γ_θ, and the later character in the second.

A concrete parametrization is

$$\Gamma_\theta(s, a) = (L_\theta s) \odot (R_\theta a),$$

where $L_\theta, R_\theta \in \mathbb{R}^{m \times d}$ are trainable matrices and $\odot$ denotes coordinatewise multiplication. The recurrence becomes

$$s_{n+1} = s_n + a_{n+1}, \qquad c_{n+1} = c_n + (L_\theta s_n) \odot (R_\theta a_{n+1}).$$

This is a recurrent architecture with multiplicative interactions and parameters shared across stages. It is related in this respect to multiplicative recurrent networks [SMH11, Section 3], although the particular accumulation rule above is not the architecture defined in that paper. We refer to it here as a *bilinear accumulation RNN*. One can also add nonlinearity and normalization here so that the structure is potential more generalizable in practice, but here we emphasize on the structure correspondence with our formalization.

The axioms of the knowledge world can be checked directly. Since $\Gamma_\theta(s, 0) = 0$, the null-character updates satisfy

$$\mu_1^0(s) = (s, 0), \qquad \mu_n^0(s, c) = (s, c) \quad (n \geq 2).$$

Together with $\mu_0^0(*) = 0$, these are the natural inclusions between successive state spaces. Consequently, inserting a null character leaves the encoded information unchanged, and the world is placeholder-free.

To examine the abelian condition, let

$$\Phi_a(s,c) = \big(s+a, c+\Gamma_\theta(s,a)\big)$$

denote the update on H. Then

$$\Phi_b(\Phi_a(s,c)) - \Phi_a(\Phi_b(s,c)) = \big(0, \Gamma_\theta(a,b) - \Gamma_\theta(b,a)\big).$$

The same calculation applies at the initial stages under the inclusions above. Hence this knowledge world is abelian if and only if Γ_θ is symmetric. A nonsymmetric pairing can therefore preserve information about input order, while still satisfying the placeholder-free condition. It does not, however, retain the entire sequence: the state records only the displayed first-order and second-order summaries.

3.4.2. *A nonlinear architecture: the vanilla RNN.* We now use the same state spaces with a different interpretation and update rule. For $n \geq 2$, a state $(s,h) \in V \oplus U$ stores the most recent character s and a hidden representation h of the preceding computation.

Fix an initial hidden state $h_0 \in U$, trainable matrices

$$W_{\text{in}} \in \mathbb{R}^{m\times d}, \qquad W_{\text{rec}} \in \mathbb{R}^{m\times m},$$

and a bias $b \in U$. Choose a continuous activation σ, such as tanh, applied coordinatewise. Define

$$F_\theta : U \times V \to U, \qquad F_\theta(h,a) = \sigma(W_{\text{rec}}h + W_{\text{in}}a + b).$$

In this construction, intelligence evaluates the complete next hidden representation, rather than an increment to the existing memory. Set

$$\begin{aligned} w_0(*)(a) &= F_\theta(h_0, a), \\ w_1(s)(a) &= F_\theta(F_\theta(h_0,s), a), \\ w_n(s,h)(a) &= F_\theta(h,a), \qquad n \geq 2. \end{aligned}$$

The formula for w_1 accounts for the special state space $A_1 = V$: after the first character s, the hidden representation is determined by s as $F_\theta(h_0,s)$ and need not be independently stored.

For $n \geq 2$, the associated attention pairing is

$$B_n((s,h),a) = \sigma(W_{\text{rec}}h + W_{\text{in}}a + b).$$

This pairing is generally nonlinear. Here, as in our general definition, "pairing" means a structured two-input evaluation map and does not imply bilinearity.

Define the knowledge updates by

$$\mu_0(a,*) = a, \qquad \mu_n(a,x) = \big(a, w_n(x)(a)\big) \quad (n \geq 1).$$

In particular, for $n \geq 2$,

$$\mu_n(a,(s,h)) = \big(a, \sigma(W_{\text{rec}}h + W_{\text{in}}a + b)\big).$$

The first component is replaced by the new character, and the second is replaced by the hidden representation evaluated by the intelligence.

Given characters $a_1, \ldots, a_N$, define

$$h_n = F_\theta(h_{n-1}, a_n) = \sigma(W_{\text{rec}}h_{n-1} + W_{\text{in}}a_n + b).$$

The corresponding knowledge states are

$$x_1 = a_1, \qquad x_n = (a_n, h_n) \quad (n \geq 2).$$

This is precisely the hidden-state recurrence of a *vanilla recurrent neural network*; see [SMH11, Section 2]. The parameters are shared across all stages. The additional V component retains the most recent character, while the usual recurrent memory is carried by U.

Unlike the bilinear accumulation model, this knowledge world is not placeholder-free when $V \neq \{0\}$. Indeed, for $n \geq 2$,

$$\begin{aligned} \mu^0_{n+1}\big(\mu^a_n(s,h)\big) &= \big(0, F_\theta(F_\theta(h,a),0)\big), \\ \mu^a_{n+1}\big(\mu^0_n(s,h)\big) &= \big(a, F_\theta(F_\theta(h,0),a)\big). \end{aligned}$$

For $a \neq 0$, these states already differ in their first components. Hence this realization is also nonabelian. Moreover, even when considering only the hidden component, a zero input is not generally an inactive step, since

$$F_\theta(h,0) = \sigma(W_{\text{rec}}h + b)$$

need not equal h. Ignoring padding would therefore require an additional masking rule, rather than merely encoding padding by the zero vector.

4. Geometric spaces that lead to new designs

As we discussed above, the knowledge world and intelligence could leads to the design of architecture, as showed in table 1.

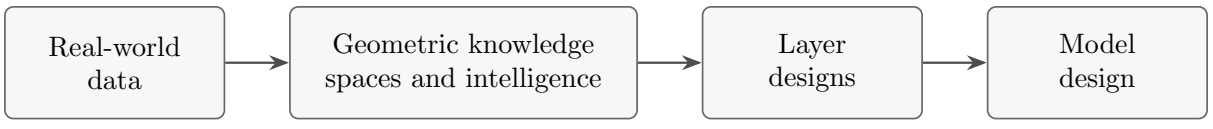
Architecture	**knowledge worlds**	**Intelligence**	**Update and readout**
Attention (ordinary or causal)	$A = E \oplus E \oplus F$, $a = (q, k, u)$, $A_n = A^n$, $X = (a_1, \dots, a_n)$.	$V_n = \mathbb{R}^n$, $B_n(X, a) = (s_j)_{j=1}^n$, $s_j = \langle k_j, q \rangle$.	$\mu_n(a, X) = (X, a)$. $\alpha = \mathrm{sm}(s_J/\sqrt{r})$, $y = \sum_{j \in J} \alpha_j u_j$.
Linear attention	$A = \mathcal{S}_1$, $A_n = \mathcal{S}_n$, $H = [z \mid M]$, $a = [q \mid qu^{\mathsf{T}}]$.	$V_n = \mathbb{R}^2$, $B_n(H, a) = (s, c)$, $s = u^{\mathsf{T}} M^{\mathsf{T}} q$, $c = z^{\mathsf{T}} q$.	$\mu_n(a, H) = H + a$. $y = \frac{M^{\mathsf{T}} q}{\varepsilon + z^{\mathsf{T}} q}$. Normalized evaluation: $\frac{s}{\varepsilon + c} = \langle y, u \rangle$.
Tensor attention	$A = \mathcal{T}$, $a = (Q, K, U)$, $A_n = A^n$, $X = (a_1, \dots, a_n)$.	$V_n = \mathbb{R}^{h \times n}$, $[B_n(X, a)]_{\ell j} = \langle Q_\ell, K_{j,\ell} \rangle$.	$\mu_n(a, X) = (X, a)$. $P = \mathrm{sm}_{\mathrm{row}}(B_n/\sqrt{r})$, $y_\ell = \sum_{j=1}^n P_{\ell j} U_{j,\ell}$. Concatenate heads and apply W_O.
Bilinear RNN	$A = \mathbb{R}^d$, $A_1 = A$, $A_n = A \oplus F$, $x = (s, c)$.	$V_n = F$, $B_n(x, a) = \Gamma_\theta(s, a)$. The evaluation is an increment to memory.	$s' = s + a$, $c' = c + \Gamma_\theta(s, a)$, $\mu_n(a, x) = (s', c')$.
Vanilla RNN	$A = \mathbb{R}^d$, $A_1 = A$, $A_n = A \oplus F$, $x = (s, h)$.	$V_n = F$, $B_n(x, a) = f_\theta(h, a)$. The evaluation is the next hidden state.	$h' = f_\theta(h, a)$, $\mu_n(a, x) = (a, h')$. The hidden state is replaced by its evaluation.

Table 1. knowledge worlds, intelligence pairings, and the resulting neural architectures. The RNN formulas describe the stabilized stages $n \geq 2$, with initial stages as defined in the preceding constructions.

An inspiring aspect of this correspondence is the possibility of designing new neural network architectures by constructing suitable geometric spaces. Topology, differential geometry, and algebraic geometry offer a wealth of structures to explore. This framework therefore provides a potential route for applying the objects and tools of modern mathematics to neural network design.

A possible design strategy within this framework proceeds as follows. Given a learning task involving data such as text, music, images, or motion, we first identify the relevant structural features of the data and choose geometric models that reflect them. For example, when the task does not depend on input order, abelian knowledge worlds provide a natural starting point. We then equip the chosen knowledge worlds with suitable update maps and intelligence, expressed through attention pairings, and use these structures to construct and test individual layers. Finally, we assemble promising layers into a model, specifying how information is passed between them.

The process can thus be summarized as



For mathematicians, a natural problem is to construct geometric systems (A_n, μ_n, w_n) satisfying the relevant axioms. Each such system may suggest a candidate for a new attention-based architecture, whose computational feasibility and effectiveness can then be investigated.

We now present several geometric constructions satisfying our axioms. The design of attention layers based on these constructions is left for future work.

4.1. **Further geometric knowledge worlds.**

We begin with rank filtrations in real matrix spaces.

Example 4.1. The knowledge world underlying linear attention is an augmented version of the geometric knowledge world in Example 2.3, in which nonzero words are represented by rank-one matrices in $\mathfrak{gl}(N,\mathbb{R})$. This suggests analogous constructions using other real Lie algebras.

For instance, define

$$A = \{uv^\top - vu^\top : u, v \in \mathbb{R}^d\},$$

and

$$A_s = \{K \in \mathfrak{so}(d,\mathbb{R}) : \operatorname{rank} K \le 2s\}, \qquad s \ge 0.$$

Then $A_0 = \{0\}$ and $A_1 = A$. The knowledge updates are given by addition:

$$\mu_s : A \times A_s \longrightarrow A_{s+1}, \qquad (a, K) \longmapsto K + a.$$

In particular, the action of the empty word is the inclusion $A_s \hookrightarrow A_{s+1}$.

Fix a map

$$F : \mathfrak{so}(d,\mathbb{R}) \longrightarrow \mathfrak{so}(d,\mathbb{R})$$

of the regularity required by the chosen category. The simplest choice is $F = \mathrm{id}$. We then define the attention pairing by

$$w_n : A_n \times A \longrightarrow \mathbb{R}, \qquad w_n(K, b) = -\frac{1}{2}\operatorname{tr}(F(K)b).$$

Analogous constructions arise from rank filtrations on symmetric matrices, the symplectic Lie algebra, and the special linear Lie algebra.

Example 4.2 (Symmetric matrices)**.** Let $\mathrm{Sym}_d(\mathbb{R})$ denote the space of real symmetric $d \times d$ matrices. Define

$$A = \{c\,uu^\top : c \in \mathbb{R},\ u \in \mathbb{R}^d\},$$

and

$$A_s = \{S \in \mathrm{Sym}_d(\mathbb{R}) : \operatorname{rank} S \le s\}, \qquad s \ge 0.$$

Then $A_0 = \{0\}$ and $A_1 = A$. The knowledge updates are

$$\mu_s : A \times A_s \longrightarrow A_{s+1}, \qquad (a, S) \longmapsto S + a.$$

Fix a map

$$F : \mathrm{Sym}_d(\mathbb{R}) \longrightarrow \mathrm{Sym}_d(\mathbb{R})$$

of the regularity required by the chosen category, for example $F = \mathrm{id}$. Define the attention pairing by

$$w_n : A_n \times A \longrightarrow \mathbb{R}, \qquad w_n(S, b) = \operatorname{tr}(F(S)b).$$

Example 4.3 (The special linear Lie algebra)**.** Define

$$A = \{uv^\top : u, v \in \mathbb{R}^d,\ v^\top u = 0\},$$

and

$$A_s = \{H \in \mathfrak{sl}(d,\mathbb{R}) : \operatorname{rank} H \le s\}, \qquad s \ge 0.$$

Then $A_0 = \{0\}$ and $A_1 = A$. Every word $a \in A$ satisfies $a^2 = 0$. The knowledge updates are

$$\mu_s : A \times A_s \longrightarrow A_{s+1}, \qquad (a, H) \longmapsto H + a.$$

Fix a map

$$F : \mathfrak{sl}(d,\mathbb{R}) \longrightarrow \mathfrak{sl}(d,\mathbb{R})$$

of the regularity required by the chosen category, for example $F = \mathrm{id}$. Define the attention pairing by

$$w_n : A_n \times A \longrightarrow \mathbb{R}, \qquad w_n(H, b) = \operatorname{tr}(F(H)b).$$

Example 4.4 (The symplectic Lie algebra)**.** Let

$$J = \begin{pmatrix} 0 & I_d \\ -I_d & 0 \end{pmatrix}, \qquad \mathfrak{sp}(2d,\mathbb{R}) = \{H \in M_{2d}(\mathbb{R}) : H^\top J + JH = 0\}.$$

Define

$$A = \{c\,uu^\top J : c \in \mathbb{R},\ u \in \mathbb{R}^{2d}\},$$

and

$$A_s = \{H \in \mathfrak{sp}(2d,\mathbb{R}) : \operatorname{rank} H \le s\}, \qquad s \ge 0.$$

Then $A_0 = \{0\}$ and $A_1 = A$. Every word $a \in A$ satisfies $a^2 = 0$. The knowledge updates are

$$\mu_s : A \times A_s \longrightarrow A_{s+1}, \qquad (a, H) \longmapsto H + a.$$

Fix a map

$$F : \mathfrak{sp}(2d,\mathbb{R}) \longrightarrow \mathfrak{sp}(2d,\mathbb{R})$$

of the regularity required by the chosen category, for example $F = \mathrm{id}$. Define the attention pairing by

$$w_n : A_n \times A \longrightarrow \mathbb{R}, \qquad w_n(H, b) = -\operatorname{tr}(F(H)b).$$

Example 4.5 (Symmetric products)**.** Let $A = \mathbb{R}^d$, pointed at the origin, and define

$$A_s = \mathrm{Sym}^s(A) := A^{\times s}/\mathfrak{S}_s, \qquad s \geq 0,$$

with the quotient topology. Here A_0 is a point and $A_1 = A$. The inclusions append the empty word:

$$[a_1, \ldots, a_s] \longmapsto [a_1, \ldots, a_s, 0].$$

The knowledge updates are

$$\mu_s : A \times A_s \longrightarrow A_{s+1}, \qquad \big(a, [a_1, \ldots, a_s]\big) \longmapsto [a_1, \ldots, a_s, a].$$

Define the attention pairing by

$$w_n : A_n \times A \longrightarrow \mathbb{R}, \qquad w_n([a_1, \ldots, a_n], q) = \prod_{j=1}^{n} (1 + a_j^\top q) - 1.$$

This expression is invariant under permutations of the a_j and hence defines a continuous pairing on $A_n \times A$.

Example 4.6 (Projective polynomial spaces)**.** We work in the smooth category. Let

$$A = \mathbb{CP}^1 = \mathbb{P}(\mathbb{C}^2), \qquad 0_A = [e_0],$$

where $e_0 \in \mathbb{C}^2$ is a fixed nonzero vector, and define

$$A_s = \mathbb{P}(\mathrm{Sym}^s \mathbb{C}^2), \qquad s \geq 0.$$

Then A_0 is a point and $A_1 = A$. The knowledge updates are given by symmetric multiplication:

$$\mu_s : A \times A_s \longrightarrow A_{s+1}, \qquad ([u], [P]) \longmapsto [P \odot u].$$

The inclusion $A_s \hookrightarrow A_{s+1}$ is

$$[P] \longmapsto [P \odot e_0].$$

Equip $\mathbb{C}^2$ with a Hermitian inner product and its symmetric powers with the induced inner products. Define the attention pairing by

$$w_n : A_n \times A \longrightarrow \mathbb{R}, \qquad w_n([P], [q]) = \frac{|\langle P, q^{\otimes n} \rangle|^2}{\|P\|^2 \|q\|^{2n}}.$$

This expression is independent of the choices of homogeneous representatives and defines a smooth real-valued pairing.

References

[BBCV21] Michael M. Bronstein, Joan Bruna, Taco Cohen, and Petar Veličković, *Geometric deep learning: Grids, groups, graphs, geodesics, and gauges*, arXiv preprint arXiv:2104.13478 (2021).

[BCB15] Dzmitry Bahdanau, Kyunghyun Cho, and Yoshua Bengio, *Neural machine translation by jointly learning to align and translate*, International Conference on Learning Representations, 2015.

[KVPF20] Angelos Katharopoulos, Apoorv Vyas, Nikolaos Pappas, and François Fleuret, *Transformers are RNNs: Fast autoregressive transformers with linear attention*, Proceedings of the 37th International Conference on Machine Learning, Proceedings of Machine Learning Research, vol. 119, PMLR, 2020, pp. 5156–5165.

[SMH11] Ilya Sutskever, James Martens, and Geoffrey Hinton, *Generating text with recurrent neural networks*, Proceedings of the 28th International Conference on Machine Learning, 2011, pp. 1017–1024.

[VSP+17] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin, *Attention is all you need*, Advances in Neural Information Processing Systems, vol. 30, 2017.

[ZKR+17] Manzil Zaheer, Satwik Kottur, Siamak Ravanbakhsh, Barnabás Póczos, Ruslan Salakhutdinov, and Alexander J. Smola, *Deep sets*, Advances in Neural Information Processing Systems, vol. 30, 2017.

[ZLY+25] Yifan Zhang, Yifeng Liu, Huizhuo Yuan, Zhen Qin, Yang Yuan, Quanquan Gu, and Andrew Chi-Chih Yao, *Tensor product attention is all you need*, arXiv preprint arXiv:2501.06425 (2025).